E-Commerce Database

ST207 AT2026 Assignment1 Submission

by

Matthew Ryan (60346)

Bachelor of Science

in

Mathematics with Data Science

London School of Economics and Political Science
London, United Kingdom



© 2026
Matthew Ryan (60346)

1 Conceptual / ER-Modelling

My E-R model broke down the core components that would be needed inside of an E-Commerce companies' database. I modelled objects inside the database as entities, and actions between these entities as relationships. For example, two of my entities were CUSTOMER and REVIEW, which were related by the action: reviews which was the relationship between the two.

Attributes like phone number and address line 2 were optional as these were not required for identifying a customer or their address. Furthermore, order total was a derived column from ORDER PRODUCT where it can be derived from the sum of the dot product between quantity and unit price for each order ID.

One caveat was the relationship between ORDER and PRODUCT. Since an order can have multiple products, and a product can be inside multiple orders, this was a M:N relationship. The fix for this was to create an associative entity called ORDER-PRODUCT, which would be a completely new table in the relational schema. I also added new attributes to ORDER-PRODUCT, such as colour and size to represent product variants.

Other than that, here are the other relationships:

Relationship	**Cardinality**	**Justification**
Customer–Review (reviews)	1:N	A customer can write many reviews, which is optional as some customers may choose to not write a review. Each review is written by exactly one customer, and this is mandatory given a review has been made.
Customer–Order (places)	1:N	A customer can place many orders, and this is optional as they may place no orders. Each order is placed by exactly one customer so this is mandatory.
Review–Product (rated)	M:1	A review can only be relating to a single product, and this has to be on a product so this is mandatory. A product can have many different reviews but can also have no reviews so this is optional.

2 Relational Modelling

There were 2 main adaptations made from the E-R model to the relational model. Firstly, the derived column order total was not included inside ORDER. This is because it can easily be derived from ORDER-PRODUCT which is what was done in one of the queries later on. Furthermore, I had to dedupe product ID and customer ID from REVIEW. This was because these were meant to form the PK for this table but the way this was represented in the E-R model meant that these columns were coming up twice each.

REVIEW

customer_id	product_id	rating	description	review_date
291	77	2	NULL	2024-01-19
29	36	1	Poor quality fabric and the stitching came loose.	2024-01-26
382	77	0	Fell apart after one wash. Very disappointed.	2024-03-15

As seen above, customer ID and product ID together form the PK for this table. This means duplicate product IDs are allowed, explaining the 1:N relationship between Customer and Review.

ORDER

order_id	customer_id	order_date	delivery_date	add_line_1	add_line_2	city	postcode	country
1	587	2024-01-01	2024-01-22	38 Abbey Grove	Flat 1B	Milton Keynes	MK14 7LH	United Kingdom
2	525	2024-01-01	2024-01-03	168 Station Grove	NULL	London	NW20 5LL	United Kingdom
3	156	2024-01-01	2024-01-16	26 Windsor Lane	NULL	Plymouth	PL1 8RN	United Kingdom

As seen above, in the address line 2 column there are some NULL values. This signals the optional constraint from the E-R model on this column, allowing NULL values to take place. Furthermore, duplicate customer IDs are allowed in this table as this is only the FK and not the PK. This is because of the 1:N relationship from Customer to Order.

3 Database Creation and DDL

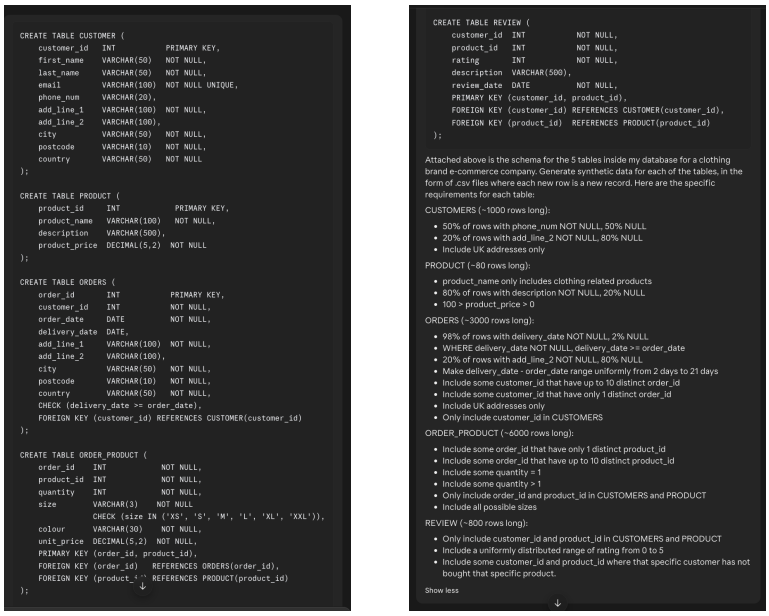
When creating the database, it was essential to create CUSTOMER and PRODUCT before any of the other tables. This was because all of the other tables included foreign

keys that came from either of these tables. When creating these, they would need to be referenced.

Some further CHECK constraints were added during this, such as making sure delivery date was bigger than or equal to the date that the order was placed. Secondly, the size that orders were placed at would need to be either XS, S, M, L, XL or XXL, making sure that sizing was consistent.

4 Data Loading

To match the schema that I had designed, I used Claude to generate synthetic data. I made sure to be specific with the prompt and handle edge cases in the data.



```
CREATE TABLE CUSTOMER (
    customer_id   INT          PRIMARY KEY,
    first_name    VARCHAR(50)  NOT NULL,
    last_name     VARCHAR(50)  NOT NULL,
    email         VARCHAR(100) NOT NULL UNIQUE,
    phone_num     VARCHAR(20),
    add_line_1    VARCHAR(100) NOT NULL,
    add_line_2    VARCHAR(100),
    city          VARCHAR(50)  NOT NULL,
    postcode      VARCHAR(10)  NOT NULL,
    country       VARCHAR(50)  NOT NULL
);

CREATE TABLE PRODUCT (
    product_id     INT           PRIMARY KEY,
    product_name   VARCHAR(100)  NOT NULL,
    description    VARCHAR(500),
    product_price  DECIMAL(5,2)  NOT NULL
);

CREATE TABLE ORDERS (
    order_id       INT          PRIMARY KEY,
    customer_id    INT          NOT NULL,
    order_date     DATE         NOT NULL,
    delivery_date  DATE,
    add_line_1     VARCHAR(100) NOT NULL,
    add_line_2     VARCHAR(100),
    city           VARCHAR(50)  NOT NULL,
    postcode       VARCHAR(10)  NOT NULL,
    country        VARCHAR(50)  NOT NULL,
    CHECK (delivery_date >= order_date),
    FOREIGN KEY (customer_id) REFERENCES CUSTOMER(customer_id)
);

CREATE TABLE ORDER_PRODUCT (
    order_id    INT           NOT NULL,
    product_id  INT           NOT NULL,
    quantity    INT           NOT NULL,
    size        VARCHAR(3)    NOT NULL
                CHECK (size IN ('XS', 'S', 'M', 'L', 'XL', 'XXL')),
    colour      VARCHAR(30)   NOT NULL,
    unit_price  DECIMAL(5,2)  NOT NULL,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id)   REFERENCES ORDERS(order_id),
    FOREIGN KEY (product_id) REFERENCES PRODUCT(product_id)
);
```

```
CREATE TABLE REVIEW (
    customer_id  INT          NOT NULL,
    product_id   INT          NOT NULL,
    rating       INT          NOT NULL,
    description  VARCHAR(500),
    review_date  DATE         NOT NULL,
    PRIMARY KEY (customer_id, product_id),
    FOREIGN KEY (customer_id) REFERENCES CUSTOMER(customer_id),
    FOREIGN KEY (product_id)  REFERENCES PRODUCT(product_id)
);
```

Attached above is the schema for the 5 tables inside my database for a clothing brand e-commerce company. Generate synthetic data for each of the tables, in the form of .csv files where each new row is a new record. Here are the specific requirements for each table:

CUSTOMERS (~1000 rows long):

- 50% of rows with phone_num NOT NULL, 50% NULL
- 20% of rows with add_line_2 NOT NULL, 80% NULL
- Include UK addresses only

PRODUCT (~80 rows long):

- product_name only includes clothing related products
- 80% of rows with description NOT NULL, 20% NULL
- 100 > product_price > 0

ORDERS (~3000 rows long):

- 98% of rows with delivery_date NOT NULL, 2% NULL
- WHERE delivery_date NOT NULL, delivery_date >= order_date
- 20% of rows with add_line_2 NOT NULL, 80% NULL
- Make delivery_date - order_date range uniformly from 2 days to 21 days
- Include some customer_id that have up to 10 distinct order_id
- Include some customer_id that have only 1 distinct order_id
- Include UK addresses only
- Only include customer_id in CUSTOMERS

ORDER_PRODUCT (~6000 rows long):

- Include some order_id that have only 1 distinct product_id
- Include some order_id that have up to 10 distinct product_id
- Include some quantity = 1
- Include some quantity > 1
- Only include order_id and product_id in CUSTOMERS and PRODUCT
- Include all possible sizes

REVIEW (~800 rows long):

- Only include customer_id and product_id in CUSTOMERS and PRODUCT
- Include a uniformly distributed range of rating from 0 to 5
- Include some customer_id and product_id where that specific customer has not bought that specific product.

Show less

Claude prompt used to generate synthetic data.

For example, I made sure to include a variety of rows that had optional columns as NULL and some with values in them, to demonstrate the optionality constraint on the columns.

Furthermore, I made sure the data generated was consistent. I made sure that all of the data was relevant in products and only represented clothing items. I made sure delivery date was always bigger than or equal to order date, matching the CHECK constraint in my DDL.

I included a variety of data where some customers only placed one order, and others placed multiple. I also included reviews placed by customers that had not actually bought a product to represent fake reviews which could be an interesting edge case.

5 SQL Programming

5.1 Order Basket Analysis

The rationale behind this query was for the database user to examine the products that people are ordering inside the most expensive orders. This could be used to identify the taste of high spending customers and create promotions tailored towards them.

First, I started off by calculating the total cost of each order by using a SUM and a GROUP BY. Then, I found a list of items and their quantities in each order as a string and used a JOIN on the two tables. Finally, I ordered by total cost for the user to see the items in the most expensive orders first.

order_id	order_basket	order_total
2946	(Pocket T-Shirt, Olive, 1), (Lambswool V-Neck Jumper, Olive, 3), (Quarter-Zip Sweatshirt, Lilac, 3), ...	864.87
2110	(Merino Crew Neck Jumper, Red, 2), (Cable Knit Jumper, Cream, 3), (Lambswool V-Neck Jumper, Olive, 1), ...	856.82
2437	(Chunky Roll Neck Jumper, Rust, 1), (Skinny Jeans, Light Wash, 2), (Tailored Trousers, Camel, 2), ...	825.89

5.2 Best Average Ratings (Excluding Fake Reviews)

The rationale behind this query was to evaluate the reviews for each product when not taking fake reviews into account.

The first thing I did was filter out all of the reviews which I identified as fake. These were reviews either written by a customer that had NOT purchased a specific product, of they made the review before the product arrived. I did this by finding the pairs of customer IDs and product IDs that matched in ORDER-PRODUCT and used an anti join to filter them out from reviews, along with reviews that were on a date less than delivery date. With this output, I created a view including all fake reviews.

Then, using the view, I used another anti join with my REVIEW table to identify all legit reviews. After this, I grouped by product ID and found the average rating for

each product, rounded to 2 decimal places. I ordered these in descending order for the user to see the best ranked products first.

product_id	product_name	description	avg_rating
20	Cropped Cardigan	Cropped fit cardigan with pearl-effect buttons.	4.00
71	Ribbed Vest Top	Fitted ribbed cotton vest top.	3.64
70	Thermal Base Layer Top	NULL	3.50

5.3 Returning Customer Details

Finding returning customers would be valuable to a business because these represent the customers that genuinely rate the product. Getting the contact details of these customers means sending an advertisement or a promotion to them would be easier.

The way that I found returning customers was by using LAG in a window function to find the previous purchase date for each customer. Any of these that were not NULL meant the customer had returned. Then, I found the distinct customer IDs and displayed their contact details.

customer_id	first_name	last_name	email	...
2	Ava	Kelly	ava_kelly@example.com	...
4	Alfie	Young	alfie.young32@example.co.uk	...
6	Charlotte	Gray	charlotte_gray@example.com	...

6 Triggers

Review Trigger

The `trg_check_review` trigger ensures that a customer can only review a product they have actually received. It fires `BEFORE INSERT OR UPDATE` on `REVIEW` and calls the function `check_review_valid()`, which joins `ORDERS` and `ORDER_PRODUCT` to check whether at least one order exists for the same `customer_id` and `product_id`, with a `delivery_date` on or before the `review_date`. If no such order exists, the operation is rejected with `RAISE EXCEPTION`; otherwise, the row is accepted. A trigger is needed rather than a `CHECK` constraint, because the rule depends on data stored in other tables. The trigger also fires on `UPDATE`, which prevents a valid review from later being

changed to refer to a product the customer never bought. Since the check only requires one valid delivered order, a customer who has bought the same product more than once is handled correctly.

The following cases were tested:

1. **Pass:** a customer reviews a product they bought, dated after delivery.
2. **Fail:** a customer reviews a product they never bought.
3. **Edge case, fail:** the customer bought the product, but the review is dated before delivery.
4. **Edge case, fail:** the customer ordered the product, but it has not yet been delivered (`delivery_date` is `NULL`).
5. **Edge case, fail:** an `UPDATE` that changes a valid review's `product_id` to one the customer never bought.

Order Trigger

The `trg_check_order_not_delivered` trigger prevents the contents of an order from being changed once it has been delivered. It fires `BEFORE INSERT OR UPDATE OR DELETE` on `ORDER_PRODUCT` and calls the function `check_order_not_delivered()`, which uses `TG_OP` to identify the operation being performed. For `UPDATE` and `DELETE`, it checks the existing row (`OLD`) to stop a delivered line from being edited or removed. For `INSERT` and `UPDATE`, it checks the new row (`NEW`) to stop an item from being added to, or moved into, a delivered order. An order counts as delivered when its `delivery_date` is on or before `CURRENT_DATE`, so orders with a `NULL` or future delivery date can still be changed. The trigger was created after the synthetic data was loaded, so that it would not block the insertion of historical delivered orders.

The following cases were tested:

1. **Pass:** changing the quantity on an order that has not yet been delivered.
2. **Fail:** an `UPDATE` to a line on a delivered order.
3. **Fail:** a `DELETE` of a line from a delivered order.
4. **Edge case, fail:** an `INSERT` of a new item into a delivered order.
5. **Edge case, fail:** moving a line from an undelivered order into a delivered one by changing its `order_id`.

For each failing case, a `SELECT` was run afterwards to confirm the data was unchanged. Passing cases were wrapped in `BEGIN; ... ROLLBACK;` so that the change could be shown to succeed without altering the test data.